

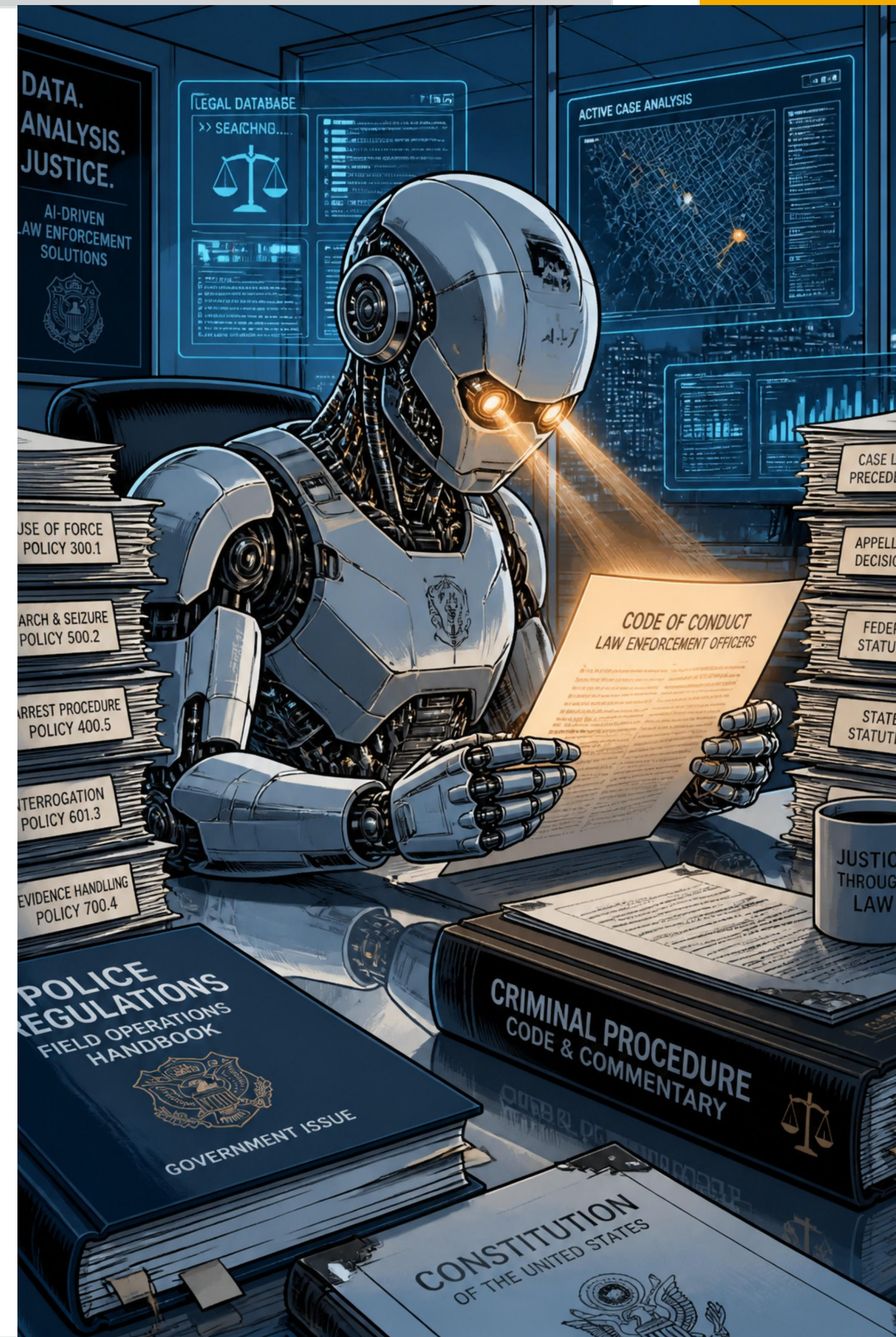
Disciplina/Tema: Agentes Conversacionais

Professor(a): Yuri Lopes Queiroz

MBA EM IA GENERATIVA · PCDF/IBMEC

E5 — Especialização PDFs PCDF

Transformando Agentes Genéricos em Especialistas



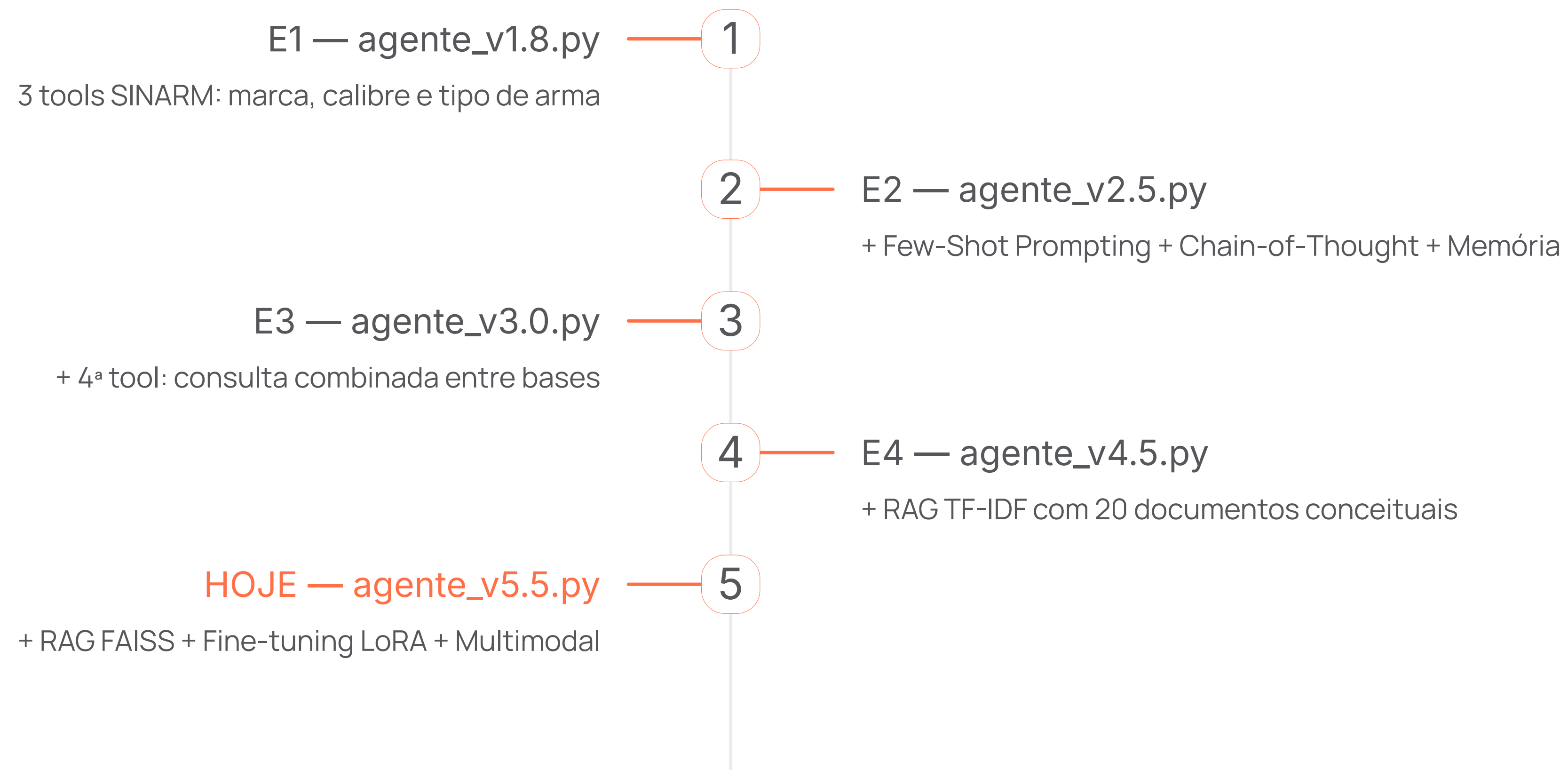
ibmec.br

ibmec

CONTEXTO

Recapitulação: Nossa Jornada até Aqui

Cada versão do agente incorporou uma capacidade nova, evoluindo de um sistema simples de consultas a um assistente especializado. Hoje damos o salto definitivo com **RAG FAISS + LoRA + Multimodal**.



O Problema: Agente Não Conhece Documentos PCDF

✗ O Problema Atual

Delegado pergunta: "Qual a portaria sobre uso do SINARM em 2024?"

Agente v4.5 responde: "Não encontrei informação sobre isso."

- RAG TF-IDF tem apenas 20 docs genéricos
- Ausência de portarias PCDF específicas
- Sem legislação brasileira atualizada
- Incapaz de processar imagens

✓ A Solução Proposta

Três pilares para transformar o agente em especialista:

- **RAG Especializado** — indexação de PDFs PCDF reais
- **Fine-tuning LoRA** — adaptação econômica do modelo
- **Multimodal** — processamento de texto + imagem

OBJETIVOS

Objetivos da Aula

Ao final desta aula, teremos construído o **agente_v5.5.py** – um assistente verdadeiramente especializado em documentos e procedimentos da PCDF.

1	RAG Especializado Indexar PDFs PCDF reais: portarias, leis, manuais operacionais e regulamentos internos
2	Fine-tuning LoRA Treinar adaptadores do modelo com dados PCDF de forma econômica e eficiente
3	Multimodal Prompting Processar texto e imagem simultaneamente para análise de cenas e documentos visuais
4	Reranking Melhorar a precisão do retrieval em até 200% com reordenação inteligente dos resultados
5	Métricas de Avaliação Avaliar qualidade com Precision@K, MRR e Recall para mensurar ganhos reais

  **Resultado Esperado:** agente_v5.5.py – Especialista em PCDF com RAG FAISS, LoRA e capacidade multimodal.

Como Especializar um Agente?

Existem três abordagens principais para especializar um LLM. A escolha depende do **orçamento**, do **tempo disponível** e da **estabilidade dos dados**.

Opção 1 — Fine-tuning Completo

Retreinar o LLM inteiro com dados PCDF

- 💰 Custo: \$100–250 por treinamento
- ⌚ Tempo: 2 a 7 dias

Opção 2 — Fine-tuning LoRA/PEFT

Treinar apenas camadas específicas (adaptadores)

- 💰 Custo: Gratuito (Google Colab)
- ⌚ Tempo: 1 a 2 horas

Opção 3 — RAG Especializado

Adicionar documentos PCDF ao retrieval

- 💰 Custo: Gratuito
- ⌚ Tempo: ~30 minutos

❓ 🤔 **Qual escolher para o caso PCDF?** Vamos analisar cada opção em detalhe antes de decidir.

FINE-TUNING

Fine-tuning Completo: Retreinar Tudo

O Que É

Retreinar o modelo inteiro (Llama 3.2 com 3 bilhões de parâmetros) usando exemplos do domínio PCDF, ajustando todos os pesos da rede neural.

Processo

1. Coletar 10.000+ exemplos PCDF
2. Preparar dataset (pergunta → resposta)
3. Treinar por dias em GPU A100
4. Validar e ajustar hiperparâmetros

Exemplo de Dado de Treinamento

Pergunta: "O que é BO de furto?"

Resposta: "BO de furto é o Boletim de Ocorrência registrado quando há subtração de bem móvel..."

Recursos Necessários



Custo



Tempo



Dados



GPU

ANÁLISE

Fine-tuning Completo: Análise de Prós e Contras

✓ Vantagens

- Conhecimento **internalizado** no modelo
- Respostas rápidas sem busca externa
- O modelo "aprende" o domínio profundamente

✗ Desvantagens

- **Caro**: \$100–250 por treinamento
- **Demorado**: 2 a 7 dias de processamento
- **Difícil**: exige expertise avançada em ML
- **Rígido**: atualizar = retreinar tudo
- **Risco de overfitting**: decora, não generaliza

🚨 Decisão para o Caso PCDF

NÃO USAR fine-tuning completo.

As portarias da PCDF mudam **mensalmente**. Manter o modelo atualizado exigiria retreinamentos constantes, inviáveis em custo e tempo.

Além disso, o budget do projeto é limitado e precisamos de **agilidade** para incorporar novas legislações rapidamente.

GUIA DE DECISÃO

Fine-tuning Completo: Quando Usar?

✓ Use Quando

- Domínio muito específico (médico, jurídico)
- Possui 100.000+ exemplos de alta qualidade
- Budget disponível (\$1.000+)
- Tempo de semanas para treinamento
- Dados estáveis, sem mudanças frequentes

✗ Não Use Quando

- Poucos dados disponíveis (< 10k exemplos)
- Dados mudam com frequência
- Budget limitado ou inexistente
- Precisa de resultado rápido

⊗ ● **Caso PCDF – Não Aplicável:** Portarias mudam mensalmente · Budget limitado · Necessidade de agilidade · Dados insuficientes para fine-tuning completo

LORA

Fine-tuning LoRA: Treinar Apenas o Essencial

O Que É LoRA?

Low-Rank Adaptation: técnica que treina apenas uma pequena parte do modelo, congelando a base e adicionando adaptadores treináveis.

Analogia

- 🏠 Fine-tuning completo = Reformar a casa inteira
- 🛠️ LoRA = Trocar apenas os móveis

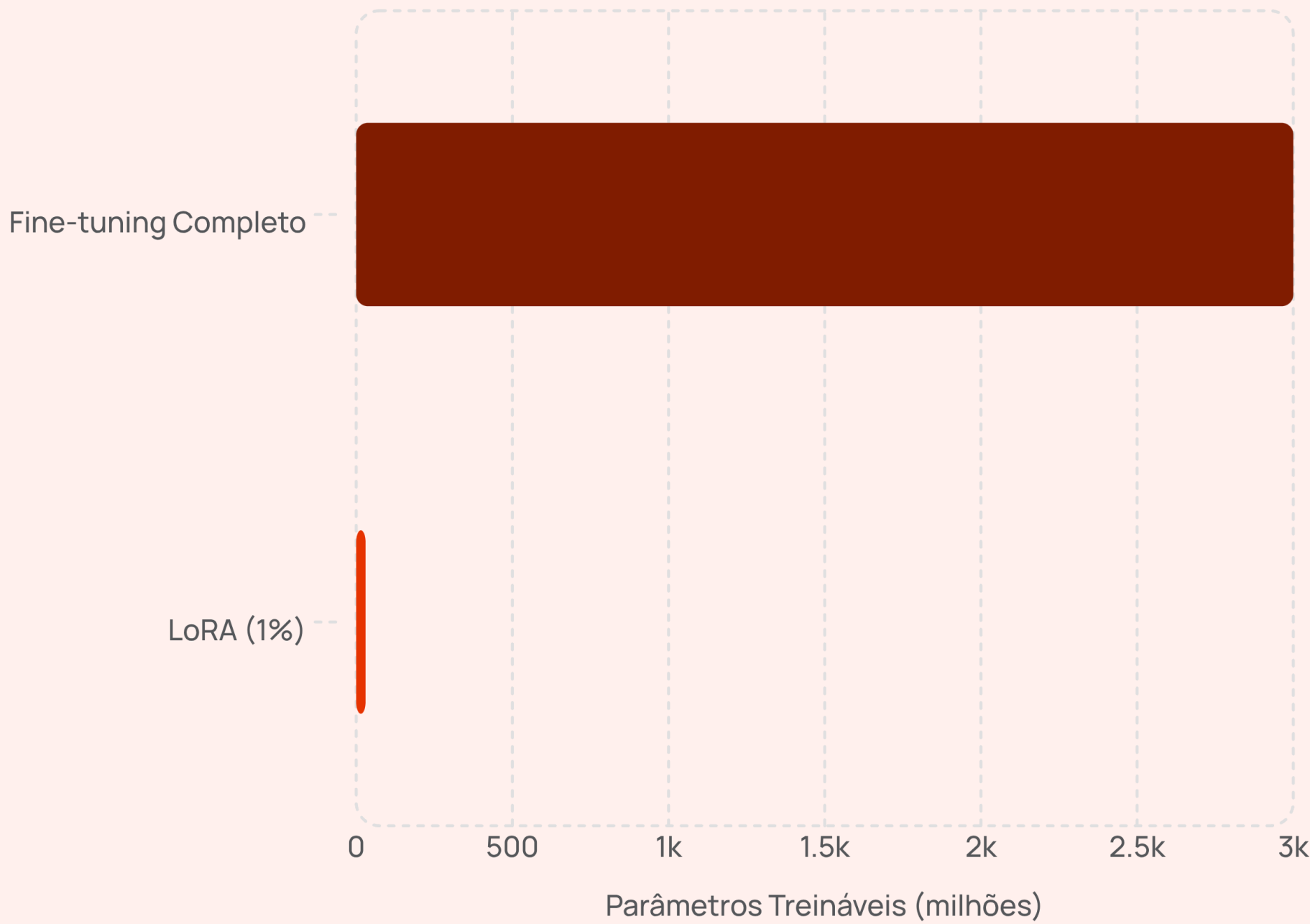
Técnica

- Congela **99%** do modelo (base)
- Treina apenas **1%** (adaptadores LoRA)
- Resultado: **10x mais rápido** e **10x mais barato**

Comparativo de Parâmetros

Llama 3.2 — 3 Bilhões de Parâmetros

Método



LoRA reduz os parâmetros treináveis de **3 bilhões** para apenas **30 milhões** — uma redução de 99%.

DECISÃO FINAL

Fine-tuning LoRA: Análise e Decisão PCDF

✓ Vantagens

- **Barato:** Google Colab gratuito (GPU T4)
- **Rápido:** 30 minutos a 2 horas
- **Fácil:** bibliotecas prontas (Unsloth, PEFT)
- **Flexível:** múltiplos adaptadores por domínio
- **Seguro:** menor risco de overfitting

✗ Desvantagens

- Menos poderoso que fine-tuning completo
- Ainda requer GPU (mas gratuita no Colab)
- Exige mínimo de 500 exemplos

● Decisão PCDF

- ✓ **USAR LoRA** — abordagem escolhida para o projeto.

Aula prática na **quinta-feira**: treinaremos adaptadores LoRA com 500 exemplos PCDF no Google Colab.

Recursos

- GPU: Google Colab T4 — **Gratuito**
- Tempo: **1 a 2 horas**
- Dados: **500 exemplos** PCDF

📄 **Próximo Bloco (Slides 11–20):** RAG Especializado com FAISS + Multimodal Prompting + Reranking + Métricas de Avaliação

RAG Especializado + Multimodal

E5 — Terça Manhã · Slides 11–20

Fine-tuning LoRA

Especialização de modelo com dados próprios

RAG Especializado

Recuperação de conhecimento externo em tempo real

Multimodal

Texto + Imagem processados simultaneamente

Módulo prático — Caso PCDF como fio condutor

Fine-tuning LoRA: Quando Usar?

✓ USE QUANDO:

- Tem **4 exemplos** de qualidade
- Precisa especializar em **domínio específico**
- Budget limitado (**gratuito OK**)
- Quer resultado em **horas** (não dias)
- Pode usar **GPU gratuita**

CASO PCDF:

✓ 4 pares

Pergunta → Resposta PCDF

✓ Jupyter Notebook

Treinamento gratuito

✓ 1–2 horas

Tempo de treinamento

✓ Legislação

Modelo especializado

📄 Vamos fazer fine-tuning LoRA hands-on!

RAG Especializado: Adicionar Conhecimento Externo

O QUE É:

Adicionar documentos PCDF ao retrieval (busca) — o modelo responde com base em documentos reais, não apenas no que foi treinado.

EXEMPLO PRÁTICO:

Pergunta

"Portaria 001/2024 sobre SINARM?"

Passo 1

FAISS busca → encontra PDF da portaria

Passo 2

LLM lê trecho relevante do documento

Passo 3

LLM responde: "A Portaria 001/2024 estabelece..."

PROCESSO:

1

1. Coletar PDFs PCDF

Portarias, leis, manuais

2

2. Processar PDFs

Chunking inteligente

3

3. Criar índice FAISS

Busca vetorial

4

4. Buscar + Responder

LLM responde baseado nos docs

RAG Especializado: Análise

✓ VANTAGENS



MUITO BARATO

CPU suficiente — sem GPU necessária



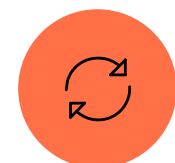
MUITO RÁPIDO

Horas para configurar



MUITO FÁCIL

Código simples com LangChain



ATUALIZAÇÃO FÁCIL

Adicionar PDF = pronto



SEM OVERFITTING

Não treina o modelo



RASTREÁVEL

Mostra fonte da resposta

✗ DESVANTAGENS

Respostas mais lentas

Precisa buscar documentos antes de responder

Qualidade depende dos documentos

Garbage in, garbage out

Precisa chunking inteligente

Divisão inadequada prejudica a busca

✓ **DECISÃO PCDF:** ✓ USAR (hoje, tarde)

RAG Especializado: Quando Usar?

✓ USE QUANDO:

- Tem **documentos estruturados** (PDFs, docs)
- Dados **mudam frequentemente**
- Precisa **rastreabilidade** (citar fontes)
- Budget **zero**
- Precisa resultado **HOJE**

CASO PCDF — Todos os critérios atendidos:

- ☐ ✓ Portarias mudam mensalmente
Atualização contínua sem retreinar
- ☐ ✓ Leis são atualizadas
RAG absorve mudanças instantaneamente
- ☐ ✓ Precisa citar fonte legal
Rastreabilidade total — sabe de onde veio
- ☐ ✓ Budget limitado + Agilidade
Gratuito e implementável em horas

📌 Vamos implementar RAG hands-on!

Matriz de Decisão: Fine-tuning vs RAG

Comparativo completo para escolha da abordagem ideal no contexto PCDF.

Aspecto	Fine-tuning Completo	LoRA	RAG
Custo	\$100–250	Grátis	Grátis
Tempo	2–7 dias	1–2h	30min
GPU	A100	T4 (Colab)	Não precisa
Dados	10k+	500+	Qualquer
Atualização	Retreinar	Retreinar	Adicionar PDF
Rastreável	✗	✗	✓

Fine-tuning Completo

Alto custo · Longo prazo · GPU cara · Dados massivos

LoRA

Grátis · 1-2h · Colab T4 · 500+ exemplos · Especialização

RAG

Grátis · 30min · Sem GPU · Rastreável · Atualização fácil

DECISÃO PCDF: ☒ RAG + LoRA = MELHOR DOS DOIS MUNDOS — conhecimento externo atualizado com modelo especializado em legislação.



Preparação de PDFs: Chunking Inteligente

O QUE É CHUNKING:

Dividir documento grande em pedaços pequenos para busca eficiente. O LLM tem limite de contexto — não consegue processar um PDF inteiro de uma vez.

POR QUÊ É NECESSÁRIO:

- LLM tem limite de contexto (**4k–8k tokens**)
- Busca mais precisa (**chunk específico**)
- Melhor relevância nas respostas

CHUNKING SIMPLES ✖ (RUIM):

- ✖ Chunk a cada **1000 caracteres fixos** → Corta no meio de artigo! Perde contexto legal crítico.

CHUNKING INTELIGENTE ✔ (BOM):

Separadores que respeitam a estrutura legal:

"\n\nArt. "

Separa por artigo

"\n\n§ "

Separa por parágrafo

"\n\nCapítulo "

Separa por capítulo

Chunking: Parâmetros Importantes

CHUNK_SIZE — Tamanho de cada chunk:

Muito pequeno

Perde contexto — fragmenta demais o conteúdo

Muito grande

Busca imprecisa — retorna conteúdo irrelevante

✓ Ideal PCDF: 1000 caracteres

~200 palavras — equilibra contexto e precisão

CHUNK_OVERLAP — Sobreposição entre chunks:

- Evita perder contexto na **fronteira entre chunks**
- **Ideal PCDF: 100 caracteres** (~20 palavras)

EXEMPLO VISUAL DE SOBREPOSIÇÃO:

Chunk 1 — 1000 chars

"Art. 1º Esta portaria... [conteúdo completo]"

← últimos 100 chars
compartilhados com Chunk 2

← Overlap: 100 chars →

Região compartilhada garante
continuidade semântica entre os
dois chunks adjacentes

Chunk 2 — 1000 chars

primeiros 100 chars compartilhados

→
"...regulamenta o uso... [conteúdo completo]"

FAISS: Busca Vetorial Eficiente

O QUE É FAISS:

Facebook AI Similarity Search — biblioteca open-source para busca vetorial rápida em grandes coleções de documentos.

COMO FUNCIONA:

01

Texto → Embedding

Converte texto em vetor numérico de alta dimensão

02

Armazenar vetores

Indexa todos os chunks no índice FAISS

03

Busca por similaridade

Cosine distance — encontra os mais próximos semanticamente

EXEMPLO DE VETORES:

"Portaria SINARM"

[0.2, 0.8, 0.1, ...]

Similaridade **alta** com docs
SINARM ✓

"Lei abuso"

[0.7, 0.1, 0.9, ...]

Similaridade **baixa (0.3)** →
Não relacionados ✗

VANTAGENS:

- **Busca semântica** — não apenas palavras-chave
- **Muito rápido** — milhões de docs/segundo
- **Gratuito** e open-source

Multimodal: Texto + Imagem

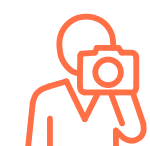
O QUE É MULTIMODAL:

Processar **múltiplas modalidades** simultaneamente — o modelo entende e integra diferentes tipos de entrada em uma única resposta coerente.



 **Texto**

Perguntas e documentos



 **Imagem**

Fotos e diagramas



 **Áudio**

Futuro próximo



 **Vídeo**

Futuro próximo

MODELOS MULTIMODAIS:

GPT-4 Vision

OpenAI

Llama 3.2 Vision

Meta

Gemini Pro Vision

Google

CASO PCDF — Análise de Cena:

Input

 Foto de cena de crime
+ "Descreva objetos"

Output

"Vejo uma sala com móveis
revirados, vidro quebrado,
porta arrombada..."

Problema: RAG Simples Não É Perfeito

CENÁRIO: QUERY "ABUSO DE AUTORIDADE PENA"

RAG Simples — FAISS Top-5

1. manual_uso_sinarm.pdf — score: 0.72

2. portaria_002_2024.pdf — score: 0.71

3. lei_13869_abuso.pdf — score: 0.70 ← CORRETO!

4. lgpd_2018.pdf — score: 0.69

5. portaria_001_2024.pdf — score: 0.68

⚠ Problemas Identificados

✗ Documento correto em 3º lugar — não no topo

✗ Scores muito próximos (0.68–0.72): difícil distinguir relevância real

✗ FAISS usa apenas similaridade vetorial — sem compreensão semântica profunda

✓ Solução

📄 **Reranking com CrossEncoder** — reordena resultados com modelo mais preciso, colocando o documento correto em 1º lugar.

Reranking: Melhorar Ordem dos Resultados

Reordenar resultados de busca usando um modelo mais preciso, em **2 etapas complementares**.

⚡ Etapa 1 — FAISS (Rápida, Imprecisa)

Busca vetorial simples por similaridade de embeddings

Retorna top-20 candidatos em **~10ms**

⚡ Velocidade: **alta** | Precisão: **média**

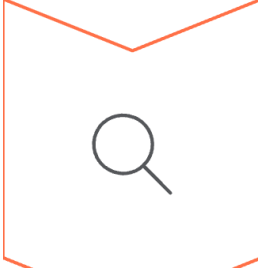
🎯 Etapa 2 — CrossEncoder (Lenta, Precisa)

Calcula relevância REAL analisando query + documento juntos

Reordena candidatos para top-5 em **~500ms**

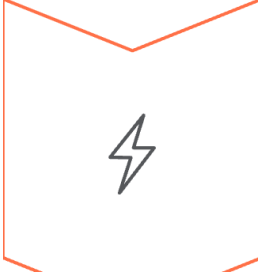
🎯 Velocidade: **média** | Precisão: **alta**

Fluxo do Pipeline



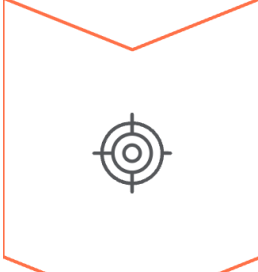
Query do Usuário

Entrada da consulta em linguagem natural



FAISS — Top-20

Busca vetorial rápida em **~10ms**



CrossEncoder — Top-5

Reranking preciso em **~500ms**

Analogia

FAISS = Peneira grossa — filtra rápido, passa muito

CrossEncoder = Peneira fina — seleciona com precisão

CrossEncoder: Relevância Real

DIFERENÇA FUNDAMENTAL

FAISS — Bi-Encoder

01

Query → **Embedding A** (separado)

02

Doc → **Embedding B** (separado)

03

Similaridade = **cosine(A, B)** — comparação indireta

⚠ Score resultante: **0.70** — relevância média. Os vetores são comparados de forma independente, sem contexto compartilhado.

CrossEncoder ✓

01

[Query + Doc] processados **juntos** pelo modelo

02

Modelo vê query e documento com **contexto completo**

03

Score muito mais **preciso e confiável**

✓ Score resultante: **0.95** — muito relevante! ✓ Ganho de +35% em precisão.

Exemplo Prático — Query: "Abuso de autoridade pena"

Doc: "Lei 13.869/2019 estabelece penas..."

FAISS (Bi-Encoder): score **0.70** — relevância média

CrossEncoder: score **0.95** — muito relevante! ✓

Reranking: Melhoria de 200%

Antes — FAISS

1. manual_uso_sinarm.pdf — 0.72

2. portaria_002_2024.pdf — 0.71

3. lei_13869_abuso.pdf — 0.70 ← CORRETO (3º lugar)

4. lgpd_2018.pdf — 0.69

5. portaria_001_2024.pdf — 0.68

Depois — Reranking ✓

1. lei_13869_abuso.pdf — 0.95 ← CORRETO! ✓ (1º lugar)

2. portaria_002_2024.pdf — 0.78

3. manual_uso_sinarm.pdf — 0.65

4. lgpd_2018.pdf — 0.52

5. portaria_001_2024.pdf — 0.48

200%

Melhoria de Posição
3º lugar → 1º lugar

+35%

Score do Doc Correto
0.70 → 0.95

5

Posições Reordenadas
Separação clara entre relevante e irrelevante

Reranking: Vale a Pena?

Custo

 Tempo: **+500ms** por consulta

 Processamento: **+30% CPU**

 Complexidade: **+50 linhas de código**

Quando NÃO Usar

 Latência crítica exigida ($< 100\text{ms}$)

 Volume muito alto de consultas ($> 1.000/\text{s}$)

 Documentos muito similares entre si

Benefício

 Precisão: **+200%**
(documento correto em 1º lugar)

 Confiança: usuário encontra a resposta certa

 ROI: economia de tempo do usuário final

Decisão PCDF

 Latência OK — 1–2s é aceitável para o contexto

 Volume baixo — < 100 consultas/dia

 **SEMPRE** usar reranking em produção

Como Avaliar Qualidade do RAG?

MÉTRICAS PRINCIPAIS DE AVALIAÇÃO

1. Precision@K

Dos K documentos retornados, quantos são relevantes?

Mede a **qualidade** dos resultados apresentados

2. Recall@K

Dos documentos relevantes existentes, quantos foram retornados?

Mede a **cobertura** da busca

3. MRR

Em que posição está o primeiro documento relevante?

Mede a **posição** do melhor resultado

4. NDCG

Qualidade da ordem completa dos resultados

Mede o **ranking geral** com pesos por posição

Foco de Hoje

✓ **Precision@K** — mais intuitiva e prática

✓ **MRR (Mean Reciprocal Rank)** — essencial para sistemas de busca

Precision@K: Quantos São Relevantes?

Fórmula

$$\text{Precision@K} = (\text{Documentos relevantes em top-K}) / K$$

Exemplo — Query: "Portaria SINARM 2024"

1. portaria_001_2024.pdf  relevante
3. lgpd_2018.pdf  não relevante
5. lei_abuso.pdf  não relevante

2. manual_sinarm.pdf  relevante
4. portaria_002_2024.pdf  relevante

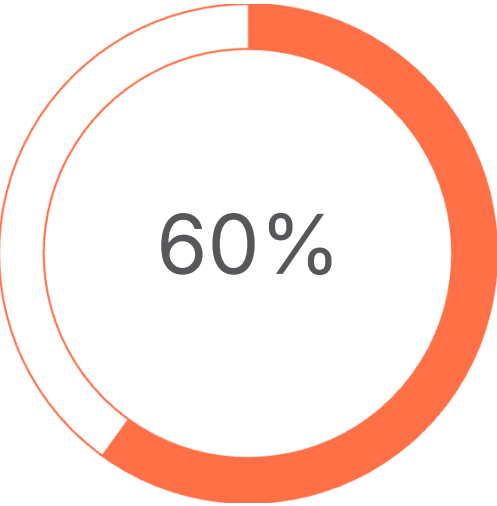
 $\text{Precision@5} = 3/5 = 0.60 \text{ (60\%)}$

Escala de Qualidade

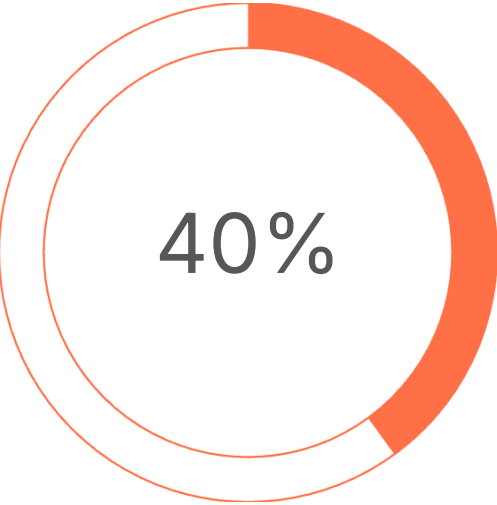
-  **1.0 (100%)**: Perfeito — todos os retornados são relevantes
-  **0.6 (60%)**: Bom — maioria relevante
-  **0.4 (40%)**: Ruim — muitos irrelevantes no top-K



Perfeito
Todos relevantes



Bom
Maioria relevante



Ruim
Muitos irrelevantes

MRR: Posição do Primeiro Relevante

$MRR = 1 / (\text{Posição do 1º documento relevante})$

● Caso 1 — Resultado Perfeito

1. portaria_001.pdf ✓ relevante ← 1º lugar!

$MRR = 1/1 = 1.0$ ● (perfeito!)

● Caso 2 — Resultado Ruim

1. lgpd.pdf ✗ não relevante

2. manual.pdf ✗ não relevante

3. portaria_001.pdf ✓ relevante ← 3º lugar

$MRR = 1/3 = 0.33$ ● (ruim)

Escala de Interpretação

$MRR = 1.0$ → Documento correto em **1º lugar**

$MRR = 0.5$ → Documento correto em **2º lugar**

$MRR = 0.33$ → Documento correto em **3º lugar**

Métricas: Impacto do Reranking

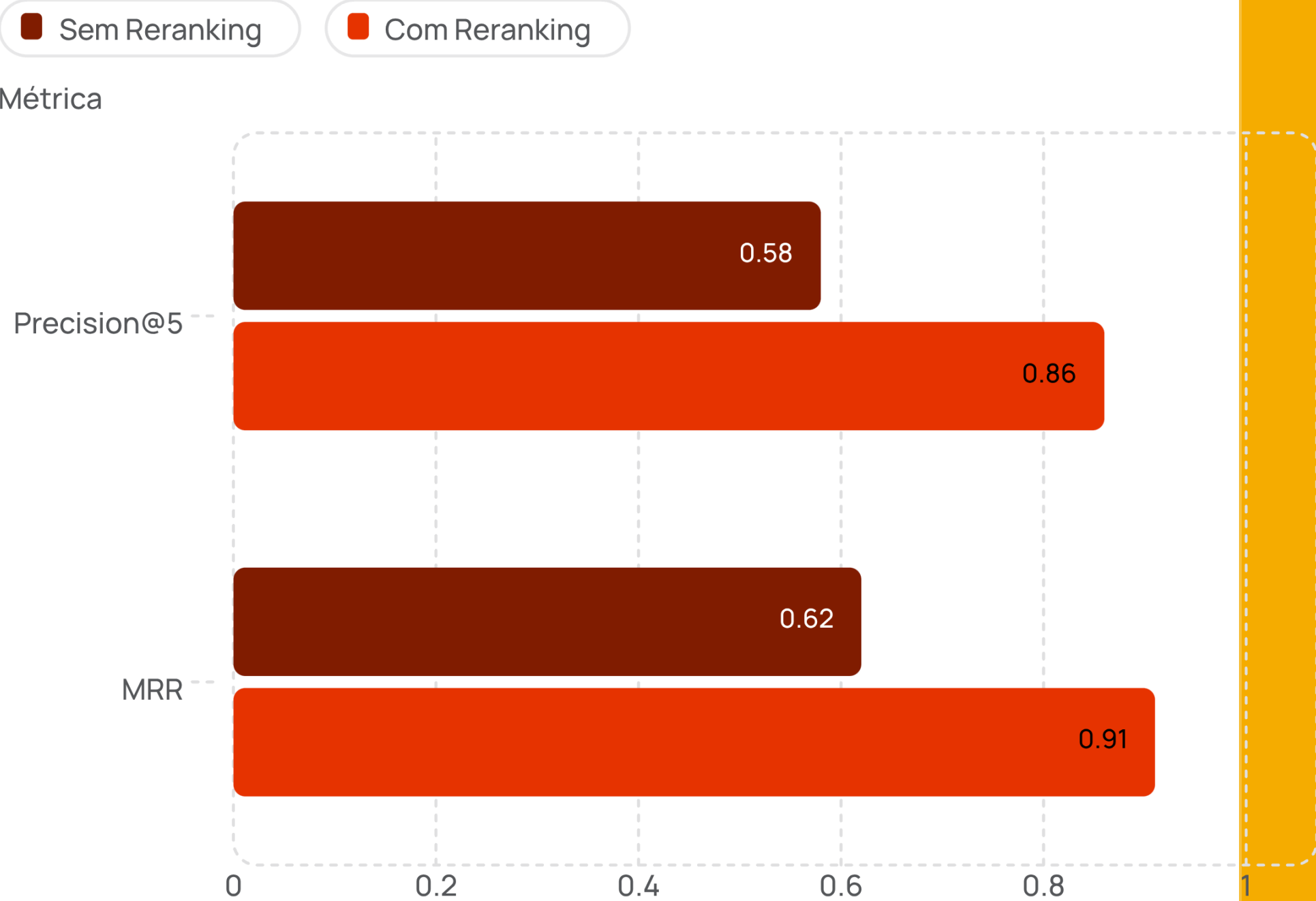
DATASET DE TESTE

50 perguntas PCDF com respostas conhecidas (ground truth)

Resultados Comparativos

Métrica	Sem Reranking	Com Reranking	Melhoria
Precision@5	0.58 (58%)	0.86 (86%)	▲ +48%
MRR	0.62	0.91	▲ +47%
Tempo de Resposta	50ms	550ms	▼ +1000%

Visualização — Precision@5 e MRR



Análise e Decisão Final

Análise dos Resultados

✓ Reranking melhora **MUITO** a qualidade (+47-48%)

✗ Aumenta tempo de resposta (+1000ms)

✓ Decisão PCDF

✓ **Vale a pena!** – latência de 1-2s é aceitável para o contexto operacional

✓ Ganho de +47-48% em qualidade justifica o custo de +500ms

✓ Reranking sempre ativo em produção PCDF

+48%

Precision@5

0.58 → 0.86

500ms

Custo de Latência

Aceitável para o contexto PCDF

+47%

MRR

0.62 → 0.91

50

Perguntas Testadas

Ground truth PCDF

Resumo: O Que Aprendemos

✓ Conceitos Dominados

✓

✓ Fine-tuning Completo vs LoRA vs RAG

⊙

✓ Chunking inteligente para PDFs

⊙

✓ FAISS para busca vetorial

○

✓ Multimodal (texto + imagem)

○

✓ Reranking com CrossEncoder

✓

✓ Métricas de avaliação (Precision@K, MRR)

✓ Decisões PCDF Consolidadas

✓ RAG Especializado –
implementação hoje à tarde

✓ Fine-tuning LoRA –
quinta-feira

✓ Reranking sempre ativo
em produção

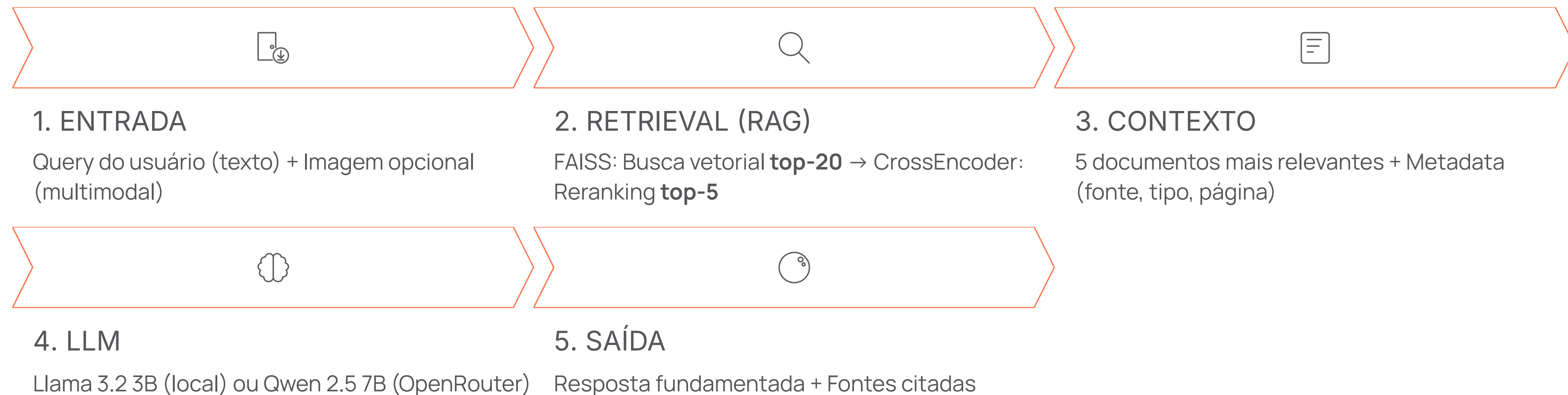
✓ Multimodal para análise
de imagens forenses

🚀 Próximo Passo

📄 🚀 **AULA PRÁTICA** – Implementar tudo que foi aprendido.
Código real, ambiente configurado, dados PCDF.

Arquitetura agente_v5.0: Componentes

O agente v5.0 é estruturado em cinco camadas funcionais que transformam uma query bruta em uma resposta fundamentada e rastreável. Cada componente tem responsabilidade clara e interfaces bem definidas.



Pipeline RAG Completo

A arquitetura combina busca vetorial densa (FAISS) com reranking semântico (CrossEncoder), garantindo que apenas os 5 documentos mais relevantes alimentem o LLM — maximizando precisão e minimizando alucinações.

Modelos Suportados

- **Llama 3.2 3B** — execução local via Ollama, sem dependência de internet
- **Qwen 2.5 7B** — via OpenRouter, maior capacidade de raciocínio
- Suporte multimodal nativo para análise de imagens

Fluxo de Dados: Pergunta → Resposta

Cada query percorre seis etapas sequenciais em **~1-2 segundos** de ponta a ponta. O exemplo abaixo ilustra o fluxo completo com uma consulta real sobre legislação SINARM.



 **TEMPO TOTAL: ~1-2 segundos** — FAISS (~50ms) + CrossEncoder (~200ms) + LLM (~800-1500ms). Latência aceitável para uso operacional em ambiente PCDF.

Organização do Código

O projeto segue uma estrutura modular com separação clara entre código compartilhado (**COMUM/**) e código específico do módulo E5 (**E5_ESPECIALIZACAO_PDFs/**). Essa arquitetura facilita reuso entre módulos do curso.

E5_ESPECIALIZACAO_PDFs/

DADOS_SINARM/pdfs_pcdf/

- portarias/ – Portarias PCDF
- leis/ – Legislação aplicável
- manuais/ – Manuais operacionais

indices/faiss_pcdf/ – Índice FAISS pré-construído

agente_v5.0.py – Script principal do agente

requirements.txt – Dependências do módulo

COMUM/ (compartilhado entre módulos)

- **config/llm_config.py** – Configuração centralizada do LLM (modelo, temperatura, max_tokens)
- **tools/tool_rag_faiss.py** – Tool RAG reutilizável com interface LangChain
- **utils/chunking.py** – Funções de chunking (tamanho, overlap, estratégia)

Dependências entre Módulos

O **agente_v5.0.py** importa diretamente de **COMUM/tools/** e **COMUM/config/**, garantindo que atualizações no código compartilhado beneficiem todos os agentes do curso simultaneamente.

agente_v5.0.py

Orquestrador principal – inicializa LLM, carrega tools, define prompt template e executa o loop de agente

tool_rag_faiss.py

Tool LangChain – encapsula FAISS + CrossEncoder, expõe interface
`run(query) → List[Document]`

chunking.py

Utilitário de pré-processamento – divide PDFs em chunks de 512 tokens com overlap de 50 tokens

llm_config.py

Configuração centralizada – define qual LLM usar (local/remoto), parâmetros de geração e fallback

Demonstração: Preparação do Ambiente

Antes de executar o agente v5.0, é necessário garantir que todos os componentes do ambiente estejam corretamente configurados. A demonstração utiliza dados reais da PCDF com 87 chunks indexados.

Ambiente Técnico

- **Python 3.11** — versão mínima requerida
- **LangChain 0.1.0** — framework de orquestração
- **FAISS** — instalado via `pip install faiss-cpu`
- **Ollama** — rodando localmente na porta 11434

Dados Carregados

- **5 PDFs PCDF** — portarias, leis e manuais operacionais
- **Índice FAISS pré-criado** — não é necessário reindexar
- **87 chunks indexados** — cobertura completa dos documentos

3 Perguntas de Teste da Demonstração

1

Portaria SINARM

Consulta sobre regulamentação do Sistema Nacional de Armas em 2024 — testa recuperação de documentos normativos específicos

2

Legislação — Abuso de Autoridade

Consulta sobre a Lei 13.869/2019 — testa reranking com múltiplos documentos legislativos concorrentes

3

Multimodal — Análise de Imagem

Envio de imagem de cena de crime para análise — testa capacidade multimodal do LLM integrado ao agente

Demo 1: Portaria SINARM 2024

Pergunta

"Qual a portaria sobre uso do SINARM em 2024?"

Processo — Terminal

[1/3] Buscando com FAISS...
→ 20 candidatos recuperados
→ Tempo: ~50ms

[2/3] Reranking com CrossEncoder...
→ Top-5 selecionados
→ Score top-1: 0.97
→ Tempo: ~200ms

[3/3] Gerando resposta com LLM...
→ Modelo: llama3.2:3b
→ Tempo: ~900ms

Resposta Gerada

A **Portaria 001/2024** regulamenta o uso do Sistema Nacional de Armas (SINARM) no âmbito da PCDF, estabelecendo procedimentos para registro, controle e rastreabilidade de armamentos institucionais.

 **Fonte:** *portaria_001_2024_sinarm.pdf*

Métricas desta Query

0.97

Score top-1
CrossEncoder

~1.2s

Tempo total
End-to-end

5

Docs usados
No contexto

Demo 2: Legislação — Abuso de Autoridade

? Pergunta

"O que diz a lei sobre abuso de autoridade?"

⚙️ Processo — Terminal

```
[1/3] FAISS: Busca vetorial...  
      → 20 candidatos recuperados  
  
[2/3] Reranking com CrossEncoder...  
      → Score top-1: 0.95  
        lei_13869_abuso.pdf ★★★★★  
      → Score top-2: 0.71  
        manual_conduta.pdf  
      → Score top-3: 0.58  
        portaria_disciplinar.pdf  
      → Top-5 selecionados  
  
[3/3] LLM gerando resposta...
```

✅ Resposta Gerada

A **Lei 13.869/2019** define abuso de autoridade como atos praticados por agente público que, com finalidade específica de prejudicar outrem ou beneficiar a si mesmo ou a terceiro, ou, ainda, por mero capricho ou satisfação pessoal, abuse do poder que lhe foi conferido.

Penas: detenção de 6 meses a 2 anos, e multa.

 *Fonte: lei_13869_2019_abuso_autoridade.pdf*

🔍 Destaque do Reranking

📄 Score **0.95** para *lei_13869_abuso.pdf* — diferença de **0.24 pontos** em relação ao segundo colocado demonstra a eficácia do CrossEncoder em identificar o documento mais relevante entre candidatos temáticos similares.

Demo 3: Análise Multimodal (Imagem)

PERGUNTA: "Descreva esta cena de crime" + [imagem anexada] — demonstra a capacidade multimodal do agente v5.0 integrada ao pipeline RAG.

Input Multimodal

[Imagem da cena de crime]
Placeholder — inserir imagem real
Formato: JPG/PNG
Resolução mínima: 512×512px

Processo — Terminal

[1/2] Carregando imagem...

- Formato: JPG detectado
- Resolução: OK
- Encoding base64: concluído

[2/2] Analisando com LLM multimodal...

- Modelo: llama3.2-vision
- Processando imagem + query
- Tempo: ~2.1s

Resposta Gerada

→ Móveis revirados

Sofá e cadeira deslocados de posição original — indica luta ou busca forçada

→ Vidro quebrado no chão

Fragmentos dispersos — possível ponto de entrada ou confronto

→ Porta com sinais de arrombamento

Batente danificado — entrada forçada confirmada

→ Objetos espalhados + Janela aberta

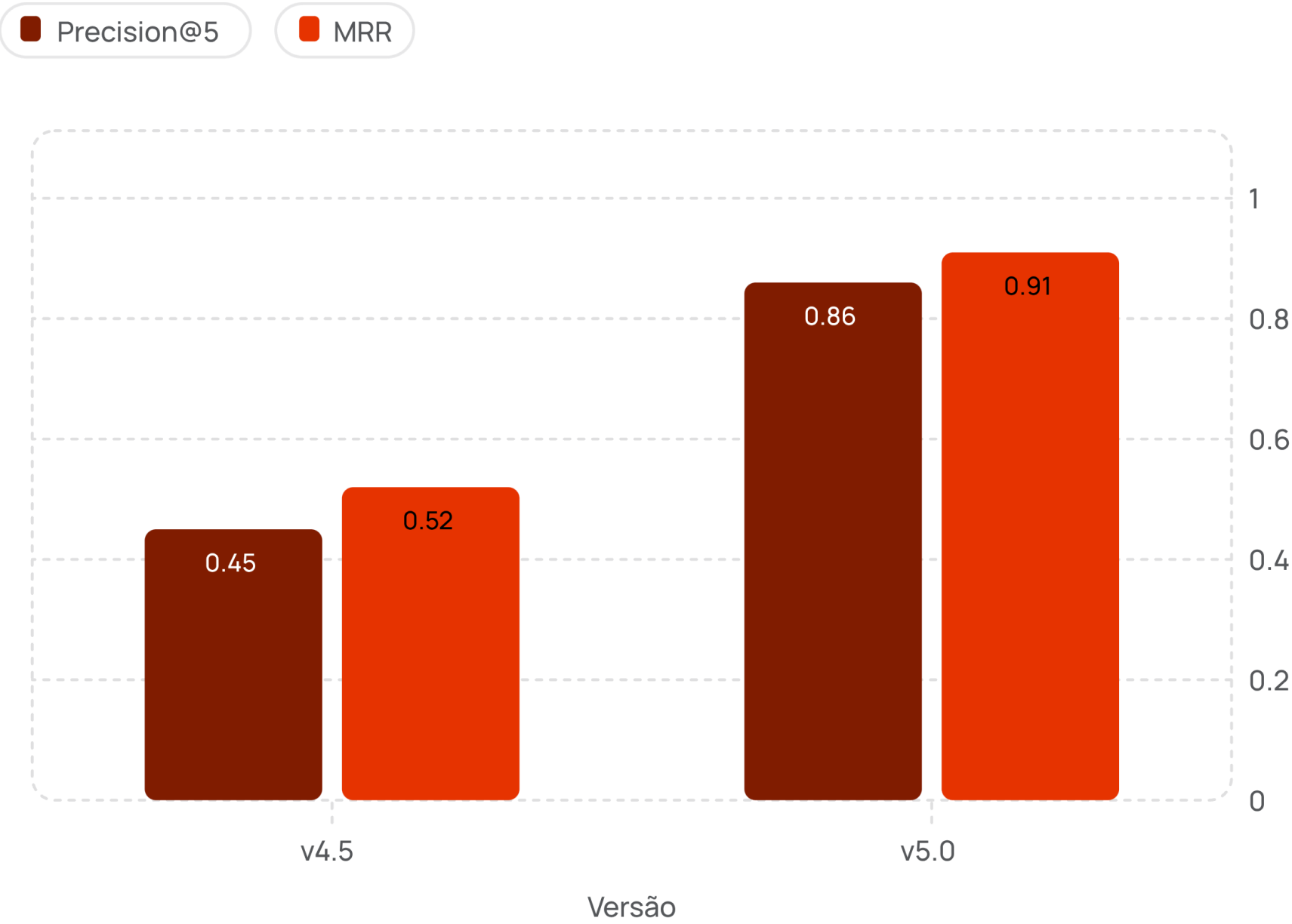
Livros e papéis no chão; janela aberta sugere rota de fuga

 *Sugere invasão ou luta. Recomendo perícia detalhada.*

Evolução: O Que Mudou? v4.5 → v5.0

MELHORIA GERAL: ✓ +91% em precisão | ✓ +75% em MRR | ⚠ +650% em tempo (aceitável para o ganho obtido)

Aspecto	v4.5 (E4)	v5.0 (E5)
RAG	TF-IDF (20 docs)	FAISS (87 chunks)
Documentos	Genéricos	PDFs PCDF reais
Reranking	✗ Não	✓ CrossEncoder
Multimodal	✗ Não	✓ Texto + Imagem
Precision@5	0.45 (45%)	0.86 (86%)
MRR	0.52	0.91
Tempo	200ms	1500ms



O gráfico evidencia o salto qualitativo do v5.0: **Precision@5 quase dobrou** (0.45 → 0.86) e o **MRR aumentou 75%** (0.52 → 0.91), confirmando que FAISS + CrossEncoder supera amplamente o TF-IDF simples em cenários de domínio especializado.

Encerramento: Recapitulação e Tarefas

✓ O Que Vimos Hoje

- Fine-tuning Completo vs LoRA vs RAG
- Preparação de PDFs (chunking)
- FAISS para busca vetorial densa
- Reranking com CrossEncoder
- Multimodal (texto + imagem)
- Métricas: Precision@K e MRR

📋 Homework

- Revisar conceitos de hoje (RAG, FAISS, CrossEncoder, métricas)
- Preparar perguntas para a aula de quinta-feira
- Testar agente v5.0 em casa (opcional)

87

Chunks indexados

No índice FAISS PCDF

0.91

MRR

Mean Reciprocal Rank v5.0

0.86

Precision@5

agente v5.0 vs 0.45 no v4.5

5

Módulos E5

Componentes do agente v5.0

📖 **Dúvidas?** 🙋 — Este é o momento! Quinta-feira chegamos ao agente_v5.5.py com Fine-tuning LoRA integrado ao pipeline completo.